

Latent Geometry Beyond Search: Amortizing Planning in World Models

Hoang Nguyen* Xiaohao Xu*† Xiaonan Huang

Department of Robotics, University of Michigan, Ann Arbor

Abstract

Modern vision-based world models can represent observations as compact yet expressive latent manifolds, but fast goal-oriented planning in these spaces remains challenging. This raises a central question: when does a learned representation simplify control, rather than merely enabling prediction? We study this question in a pretrained LeWorldModel, whose latent geometry is regularized for smoothness and uniformity. Our key insight is that, under such geometry, planning can be amortized into a latent inverse-dynamics mapping instead of requiring online search. We therefore replace iterative planning with a lightweight Goal-Conditioned Inverse Dynamics Model (GC-IDM) that maps the current latent state, goal latent state, and remaining horizon directly to the next action. Empirically, across four benchmark environments spanning navigation, contact-rich manipulation, and continuous control, our controller matches or exceeds CEM in seven of eight environment–protocol settings while reducing per-decision cost by 100–130 $\times$. A broader sweep over test-time planners (CEM, MPPI, iCEM, and gradient-based methods) shows that this result is not specific to a particular optimizer. These findings suggest that much of the structure recovered by test-time planning is already locally encoded in the latent representation. More broadly, our results indicate that sufficiently structured latent spaces can shift part of the planning burden from online optimization to learned inference.

1 Introduction

The utility of world models in control ultimately depends on their ability to facilitate efficient decision-making. Predicting future observations or latent states is useful, but it does not by itself solve the control problem: an agent must still convert those predictions into actions. In many current systems, this final step remains computationally expensive. Learned representations make forecasting accurate and fast, yet action selection still relies on substantial online optimization. As a result, there is often a gap between learning a good model of the world and using that model to act efficiently.

The LeWorldModel (LeWM) [Maes et al., 2026] is an attractive starting point because it is already efficient as a world model: with ~ 15 M parameters trainable on a single GPU in a few hours, it plans up to 48 $\times$ faster than foundation-model-based world models while remaining competitive across diverse control tasks. Yet even in LeWM, control is still implemented through CEM, which requires 9,000 candidate rollouts (45,000 predictor forward passes) for every planning step. This means that the dominant computational cost no longer comes from modeling the dynamics; it comes from searching over candidate action sequences. We refer to this mismatch between cheap prediction and expensive decision-making as the *planning tax*.

This raises a basic question: is this *tax* intrinsic, or is it primarily an artifact of how control is formulated? If the latent space is poorly organized, then search may indeed remain necessary, because

*Equal Contribution

†Project Lead

the mapping from desired future states to actions is unstable and nonlocal. But if the latent space is smooth and action-sensitive, properties that regularizers like SIGReg are designed to encourage, then action selection may admit a simpler description. In that case, planning can be viewed not as a generic combinatorial search problem, but as a local inverse problem induced by representation regularity. We study this perspective in the setting of a pretrained JEPA world model. In this paper, we replace online search with a small goal-conditioned inverse dynamics model (GC-IDM) trained on frozen LeWM latents. The model takes the current latent state, the goal latent state, and the remaining horizon, and predicts the next action directly. Our central question is *whether inverse dynamics serves as a sufficiently faithful abstraction of control for well-regularized latent representations*.

Across four benchmark environments, the proposed GC-IDM matches or exceeds CEM in seven of the eight environment–protocol cells while reducing per-decision planning cost by 100–130 $\times$. The same conclusion holds under a strict episode-level holdout, under a broad CEM compute sweep across a 500 $\times$ range, and across a broader family of test-time planners including CEM [Rubinstein, 1999], iCEM [Pinneri et al., 2021], MPPI [Williams et al., 2017], and gradient-based trajectory optimization [Hansen et al., 2022, 2024]. Together, these results suggest that, in this regime, the planner is recovering structure that the representation has already made locally accessible. On the empirical side, we confirm that a small inverse dynamics model trained by supervised regression on frozen world model latents can replace test-time search while achieving comparable or superior success rates across a broad range of benchmark conditions and maintaining high planning efficiency.

Our main contributions are: 1) We propose GC-IDM, a learned planning method for JEPA world models that recasts control as an inverse problem in latent space, replacing iterative test-time search with a single forward pass per environment step. 2) We empirically demonstrate that a lightweight goal-conditioned inverse model can outperform the broader family of test-time planners (including CEM, MPPI, iCEM, and gradient-based methods) on the LeWM benchmarks, achieve strong performance relative to prior baselines, and reduce planning cost by 100–130 $\times$. 3) Our experiments confirm that learning latent inverse dynamics on well-regularized latent representations enables fast, effective closed-loop planning.

2 Related Work

Latent world models and planning over world models. Our work sits inside a long tradition that treats a world model as something the agent plans *with*, and in that setting, the choice of planner is often where most of the inference cost resides. Latent world models learn environment dynamics in a compressed embedding space, allowing the agent to “imagine” future states and evaluate candidate behaviors through rollouts [Ha and Schmidhuber, 2018, Hafner et al., 2020, 2021, 2023]. Test-time planning is then typically implemented with model-predictive control using CEM [Rubinstein, 1999, Pinneri et al., 2021], MPPI [Williams et al., 2017], or gradient-based trajectory optimization [Hansen et al., 2022, 2024]. Prior JEPA-style world models either rely on pretrained encoders to stabilize the representation [Zhou et al., 2024, Assran et al., 2025] or use more elaborate multi-term objectives with heuristic regularization [Sobal et al., 2022, 2025]. LeWM [Maes et al., 2026] addresses both issues with a simple two-term objective based on latent future-embedding prediction and SIGReg regularization [Balestriero and LeCun, 2025], and it is the backbone we build upon. Our work is orthogonal to this line of research: we do not propose a new world model or a new optimizer, but instead ask whether the optimizer can be removed because the learned representation has effectively absorbed the computation it performs.

Inverse dynamics for control. The idea that inverse dynamics can support or replace explicit planning has been explored in several regimes, but typically as a proposal mechanism, an auxiliary prediction head, or a component of imitation learning rather than as the primary planner itself inside a JEPA latent world model. Inverse-dynamics models predict the action responsible for a state transition and have been widely used in model-based reinforcement learning and representation learning [Agrawal et al., 2016, Pathak et al., 2017, Baker et al., 2022]. GLAMOR [Paster et al., 2021] is the closest precursor: it learns a recurrent inverse-dynamics model from pixels that predicts entire action sequences conditioned on a goal state, combining the IDM with a learned action prior for planning. Our approach differs in three respects: (i) we predict a single action per step and replan from a fresh observation, rather than committing to a multi-step action sequence; (ii) our IDM is a simple feedforward MLP rather than a recurrent network; and (iii) we operate in the latent space of a

frozen pretrained JEPA world model rather than learning the representation end-to-end. Predictive inverse-dynamics methods such as Seer [Tian et al., 2025] and Latent Diffusion Planning [Xie et al., 2025] first forecast a future state and then decode an action from that prediction. By contrast, our model conditions directly on the terminal goal embedding together with the remaining horizon and relies on closed-loop re-encoding at every step, avoiding an explicit state-prediction stage inside the controller. Mimic-video [Pai et al., 2025] is especially relevant in showing that a pretrained visual backbone paired with a lightweight inverse model can yield strong control performance. Recent theory also helps explain when this strategy should work: Schäfer et al. [2026] analyze a bias–variance tradeoff in future-state-conditioned inverse dynamics, and Morin et al. [2026] argue that ground-truth inverse dynamics often lie in a lower-complexity hypothesis class than the expert policy. These perspectives are consistent with our findings that a simple inverse model can be effective when paired with a well-structured latent representation. Closely related, Luo and Du [2025] also ground a pretrained visual world model into executable actions via an inverse-dynamics-style decoder, but train it action-label-free through self-supervised goal-conditioned exploration; we instead train offline on the LeWM dataset’s action labels and use the IDM to replace the CEM optimizer entirely.

3 Preliminaries

Problem formulation. We consider finite-horizon goal-conditioned control from observations. At time t , the agent observes $\mathbf{o}_t \in \mathcal{O}$, aims to reach a goal observation $\mathbf{o}_g \in \mathcal{O}$ within a remaining horizon $h_t \in \{1, \dots, H\}$, and selects an action $\mathbf{a}_t \in \mathcal{A}$. An encoder $\text{enc}_\theta : \mathcal{O} \rightarrow \mathbb{R}^d$ maps observations to latent states $\mathbf{z}_t = \text{enc}_\theta(\mathbf{o}_t)$ and $\mathbf{z}_g = \text{enc}_\theta(\mathbf{o}_g)$. The latent-planning problem is to choose actions that minimize goal mismatch in latent space:

$$\pi^*(\mathbf{o}_t, \mathbf{o}_g, h_t) \in \arg \min_{\pi} \mathbb{E} \left[\sum_{k=0}^{h_t-1} c(\mathbf{z}_{t+k}, \mathbf{z}_g) \right], \quad (1)$$

where $c : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ is a goal-matching cost, typically induced by Euclidean distance in latent space. Search-based planners approximately solve Equation (1) online by repeatedly rolling out a learned latent dynamics model. When the latent geometry is sufficiently regular, the optimal first action may instead be recovered directly by a learned inverse map from $(\mathbf{z}_t, \mathbf{z}_g, h_t)$.

Latent world model. We consider a latent world model with an encoder enc_θ that maps observations to latent states and a predictor pred_ϕ that models one-step latent dynamics under actions:

$$\mathbf{z}_t = \text{enc}_\theta(\mathbf{o}_t), \quad \hat{\mathbf{z}}_{t+1} = \text{pred}_\phi(\mathbf{z}_t, \mathbf{a}_t). \quad (2)$$

The encoder and predictor are jointly optimized using

$$\mathcal{L}_{\text{LeWM}} \triangleq \mathcal{L}_{\text{pred}} + \lambda \text{SIGReg}(\mathbf{Z}), \quad (3)$$

Here $\mathcal{L}_{\text{pred}} = \|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2^2$ is the one-step latent prediction loss, $\hat{\mathbf{z}}_{t+1}$ is the predictor output, $\mathbf{z}_{t+1}$ is the encoded next observation, and $\mathbf{Z}$ denotes the batch of latent embeddings on which the regularizer is evaluated. The term $\text{SIGReg}(\mathbf{Z})$ is the isotropy-inducing regularizer of Balestriero and LeCun [2025], and $\lambda > 0$ is its scalar weighting coefficient relative to the prediction term. Thus, λ controls the tradeoff between predictive accuracy and regularity of the latent geometry. The geometric consequence is that the learned latent space is smooth, regularized toward well-distributed embeddings, and directly optimized for predictive dynamics.

Search-based planning in latent space. Given a current latent state $\mathbf{z}_t$ and a goal latent state $\mathbf{z}_g$, a standard planner chooses an action sequence by minimizing a terminal latent cost through repeated rollouts of the predictor:

$$\mathbf{a}_{t:t+H-1}^* \in \arg \min_{\mathbf{a}_{t:t+H-1}} d(\hat{\mathbf{z}}_{t+H}(\mathbf{a}_{t:t+H-1}), \mathbf{z}_g). \quad (4)$$

Here $d(\cdot, \cdot)$ is the latent goal-distance metric, H is the planning horizon, and $\hat{\mathbf{z}}_{t+H}(\mathbf{a}_{t:t+H-1})$ denotes the latent state predicted after rolling out the action sequence $\mathbf{a}_{t:t+H-1}$ through the learned dynamics model. In practice this optimization is usually approximated with sampling-based MPC such as CEM [Rubinstein, 1999]. With LeWM’s default configuration (300 samples, 30 refinement iterations, planning horizon 5), this comes to 9,000 candidate rollouts and 45,000 predictor forward passes per planning step. When the latent space is sufficiently regular, the optimizer in Equation (4) can be replaced by a learned inverse map.

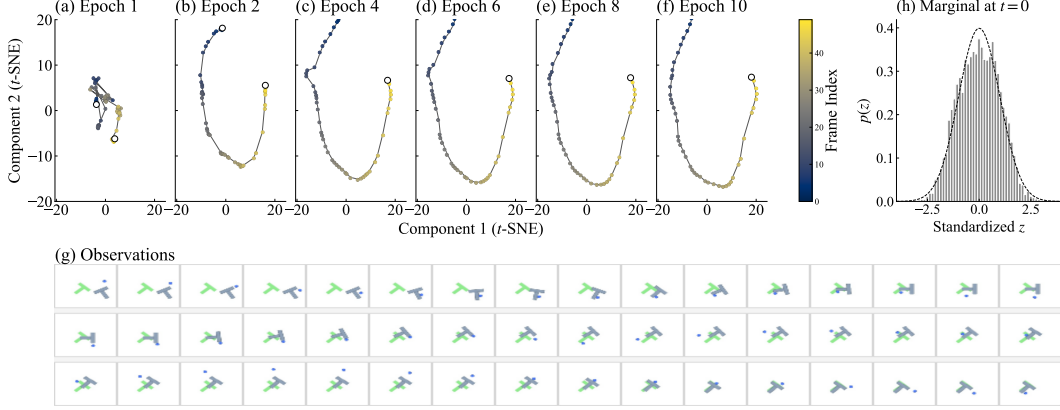


Figure 1: **Evolution of Push-T latent geometry across training.** Panels (a)–(f) show two-dimensional t-SNE embeddings of latent states from Push-T sequence at epochs 1, 2, 4, 6, 8, and 10, with points colored by frame index. Panel (g) shows subsampled observation frames from the same sequence. Panel (h) shows the standardized marginal latent distribution at $t=0$ for epoch 10, together with a Gaussian reference curve. Across training, the embedding evolves from a diffuse cloud into a smooth ordered path aligned with task progress, providing qualitative support for the claim that the learned representation is structured enough for local inverse recovery.

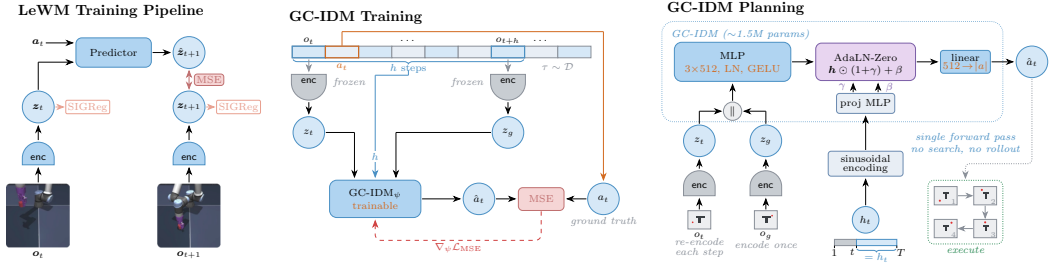


Figure 2: **Pipeline overview.** *Left:* World model encoder training process, which follows LeWM [Maes et al., 2026]. *Center:* Goal-conditioned inverse dynamics model (GC-IDM) training. From a trajectory $\tau \sim \mathcal{D}$, a tuple (z_t, z_g, h, a_t) is sampled at random horizon $h \in [1, H_{\max}]$ using frozen LeWM embeddings; the IDM is trained by MSE regression with gradients flowing only into the inverse dynamics module, i.e., GC-IDM $_{\psi}$. *Right:* GC-IDM planning. The current observation o_t is re-encoded at every step; the goal embedding z_g is encoded once and cached. The concatenated pair $z_t || z_g$ passes through a 3-layer MLP modulated by the remaining horizon h_t via AdaLN-Zero, producing estimated action $\hat{a}_t$ in a single forward inference pass with no search or rollout.

4 Method: Goal-Conditioned Inverse Dynamics Model (GC-IDM)

4.1 Overview

Motivation. Our motivation is to remove the planning tax without altering the pretrained world model. A useful failure case clarifies our design: our first attempt (Appendix D) was a *pairwise* IDM that decoded actions from consecutive latent pairs $(z_t, z_{t+1}) \rightarrow a_t$ and then planned by linearly interpolating between start and goal embeddings. That model achieved near-perfect oracle reconstruction ($R^2 = 0.993$) but poor planning. This supports the idea that bottleneck is not decoding actions from well-posed local transitions; it is constructing a valid latent path between distant states.

Latent geometry intuition. If the pretrained latent geometry is already smooth and task-aligned, then the controller does not need to invent an entire trajectory between distant states; it only needs to recover the locally appropriate action that moves the current state toward the goal under a finite budget. Figure 1 provides qualitative evidence for this picture on Push-T by showing how the t-SNE embedding of one sequence evolves across training for a pretrained LeWM, together with subsampled observations and a marginal latent histogram. Across epochs, the latent states organize from a

Algorithm 1: Closed-loop goal-reaching control induced by GC-IDM

Input: Environment $\mathcal{E}$, encoder enc_θ , inverse policy gc-idm_ψ , goal observation $\mathbf{o}_g$, horizon budget T

Output: Closed-loop action sequence $(\mathbf{a}_1, \dots, \mathbf{a}_\tau)$, where $\tau \leq T$

```
 $\mathbf{z}_g \leftarrow \text{enc}_\theta(\mathbf{o}_g)$ 
reset or initialize  $\mathcal{E}$  and observe initial observation  $\mathbf{o}_1$ 
for  $t \leftarrow 1$  to  $T$  do
     $\mathbf{z}_t \leftarrow \text{enc}_\theta(\mathbf{o}_t)$ 
     $h_t \leftarrow T - t + 1$ 
     $\mathbf{a}_t \leftarrow \text{gc-idm}_\psi(\mathbf{z}_t, \mathbf{z}_g, h_t)$ 
    apply  $\mathbf{a}_t$  to  $\mathcal{E}$  and observe next observation  $\mathbf{o}_{t+1}$ 
    if goal reached or episode terminated then
        return  $(\mathbf{a}_1, \dots, \mathbf{a}_t)$ 
    end
end
return  $(\mathbf{a}_1, \dots, \mathbf{a}_T)$ 
```

diffuse cloud into a smooth ordered path aligned with task evolution, suggesting that nearby latent displacements correspond to locally coherent behavioral changes. That is, it supports the motivating hypothesis that sufficiently regular latent geometry can make local inverse recovery easy to amortize.

Controller. Our replacement for search is, therefore, a goal-conditioned inverse dynamics operator that acts directly on the current latent state, the goal latent state, and the remaining horizon. Given observations $(\mathbf{o}_t, \mathbf{o}_{\text{goal}})$, we encode them with the frozen encoder and predict the next action by

$$\mathbf{z}_t = \text{enc}_\theta(\mathbf{o}_t), \quad \mathbf{z}_{\text{goal}} = \text{enc}_\theta(\mathbf{o}_{\text{goal}}), \quad \hat{\mathbf{a}}_t = \text{gc-idm}_\psi(\mathbf{z}_t, \mathbf{z}_{\text{goal}}, h_t). \quad (5)$$

Equation (5) is the full inference-time model. Relative to pairwise interpolation, it avoids imagined latent trajectories and conditions only on real encoder outputs from actual observations. The resulting policy is still goal-conditioned, but it shifts the problem from online search over action sequences to offline amortization of the local inverse map induced by the latent world model.

4.2 Model Architecture and Details

We purposefully instantiate Equation (5) with a lightweight neural network (Figure 2) so that any gain over CEM is attributable to latent geometry and control structure rather than raw function capacity. The backbone is a 3-layer MLP with hidden dimension 512, LayerNorm, GELU activation, and 10% dropout, with the two embeddings concatenated as $\mathbf{z}_t \parallel \mathbf{z}_{\text{goal}}$ at the input. A final linear head maps the modulated representation to the action space. Total parameters: $\sim 1.5\text{M}$, roughly 10% of the LeWM backbone and orders of magnitude smaller than the $\sim 10\text{M}$ -parameter predictor that CEM invokes 45,000 times per plan call. Given a pretrained LeWM checkpoint, GC-IDM can be trained from scratch in approximately 20 minutes per environment on a single GPU, making it a lightweight extension of an already efficient world model. The remaining-horizon variable modulates the inverse map rather than being concatenated as an undifferentiated observation feature. Concretely, the remaining-step count is first normalized as $h_t = \min(\text{steps_remaining}, H_{\max})/H_{\max} \in [0, 1]$, sinusoidally encoded (64 dimensions), and passed through a 2-layer MLP to produce an embedding $\mathbf{c}$. Two separate linear projections $\gamma(\mathbf{c}), \beta(\mathbf{c})$ then modulate the backbone output $\mathbf{h}$ via zero-initialized AdaLN-Zero modulation [Peebles and Xie, 2023]:

$$\text{AdaLN}_0(\mathbf{h}, \mathbf{c}) = \mathbf{h} \odot (\mathbf{1} + \gamma(\mathbf{c})) + \beta(\mathbf{c}), \quad (6)$$

applied immediately before the action head. The two projection layers $\gamma(\cdot), \beta(\cdot)$ have their weights and biases zero-initialized, so at the start of training $\gamma=\beta=\mathbf{0}$, ensuring the horizon signal is introduced only as the regression loss demands it.

Training on frozen embeddings. Training uses the same offline demonstration dataset that trained LeWM; no additional environment interaction is required. Each training example is induced from a trajectory by sampling (t, h) , with h drawn uniformly from $[1, H_{\max}]$, and forming the triple $(\mathbf{z}_t, \mathbf{z}_{t+h}, \mathbf{a}_t)$. The loss function is

$$\mathcal{L}_{\text{gc-idm}}(\psi) = \mathbb{E}_{(t, h) \sim \mathcal{D}} \left[\left\| \text{gc-idm}_\psi(\mathbf{z}_t, \mathbf{z}_{t+h}, h) - \mathbf{a}_t \right\|_2^2 \right]. \quad (7)$$

Table 1: **Main results.** GC-IDM vs. CEM on the four LeWM benchmark environments. Both methods use the frozen LeWM checkpoint from Maes et al. [2026] and the same solver configuration. We experiment each environment at two sample sizes: $n=200$ is our headline reporting protocol, chosen for lower variance; $n=50$ is the LeWM paper’s own protocol. Both GC-IDM and CEM are mean $\pm$ standard deviation across three seeds. Our method’s columns are underlined throughout for visual tracking; highlighted cells mark the winning method per row. Two speed up measures are recorded: *wall / ep.* is the end-to-end per-episode wall-clock ratio; *ms / plan* is the average wall-clock of a single `get_action` call (one per environment step for both methods). Because CEM buffers 25 raw actions per solver call, its average includes both expensive solver steps and cheap buffer returns; the reported 100–130 $\times$ speedup is therefore already amortized over CEM’s commit window.

Env	n	Success Rate (%)		ms / ep.		ms / plan		Speedup		Planner cost	
		Ours	CEM	Ours	CEM	Ours	CEM	wall	plan	Ours	CEM
Two-Room	50	<u>100.0$\pm$0.0</u>	82.0 $\pm$ 2.0	<u>260</u>	10486	<u>86</u>	10317	40 $\times$	120 $\times$	1.5M MLP 0.71 ms / fwd 0 pred. calls	10.8M model 45K calls / plan
	200	<u>100.0$\pm$0.0</u>	84.0 $\pm$ 2.8	<u>276</u>	10502	<u>398</u>	41318	38 $\times$	104 $\times$		
Push-T	50	<u>84.7$\pm$5.0</u>	<u>89.3$\pm$6.4</u>	<u>305</u>	10558	<u>86</u>	10340	35 $\times$	121 $\times$		
	200	<u>84.2$\pm$2.8</u>	82.5 $\pm$ 1.3	<u>322</u>	10778	<u>397</u>	42232	33 $\times$	106 $\times$		
OGB-Cube	50	<u>99.3$\pm$1.2</u>	73.3 $\pm$ 9.0	<u>8587</u>	19491	<u>84</u>	10799	2.3 $\times$	129 $\times$		
	200	<u>98.7$\pm$0.6</u>	67.0 $\pm$ 2.1	<u>9172</u>	20057	<u>334</u>	43201	2.2 $\times$	130 $\times$		
Reacher	50	<u>100.0$\pm$0.0</u>	68.0 $\pm$ 9.2	<u>1365</u>	12031	<u>93</u>	10745	8.8 $\times$	116 $\times$		
	200	<u>99.7$\pm$0.3</u>	70.3 $\pm$ 4.3	<u>1604</u>	12484	<u>399</u>	43969	7.8 $\times$	110 $\times$		

Closed-loop control at test time. At test time, the trained inverse map replaces the entire iterative optimization loop. Control is implemented as a receding-horizon inverse policy: given a goal observation $\mathbf{o}_g$ and a budget T , the controller encodes the goal once, then repeatedly re-encodes the current observation, evaluates the inverse map at the current latent state and remaining horizon, and applies the resulting action to the environment. The remaining horizon is computed as $h_t = T - t + 1$ and clamped to $H_{\max}$ (matching the training distribution); in all experiments $H_{\max} = T = 50$. There is no inner optimization loop and no open-loop commitment to a predicted latent trajectory. Algorithm 1 states the control law formally. Algorithm 1 is a closed-loop controller: after each action, it re-encodes the actual observation and recomputes the next move from the true current state. In our implementation, the added computation is a ~ 1.5 M-parameter MLP on top of encoder calls already required by LeWM. Intuitively, the latent goal distance $\|\mathbf{z}_t - \mathbf{z}_g\|$ acts as a potential field, and GC-IDM approximately follows its gradient at each step; per-step replanning accumulates these directionally correct predictions into successful trajectories.

5 Experiments

5.1 Experimental Setup

Consistent with the LeWM [Maes et al., 2026] benchmark, each evaluation environment requires goal-conditioned control from image observations, though they differ in action dimensions and contact structures: Two-Room is nearly deterministic navigation, Push-T requires extended contact-rich manipulation, OGBench-Cube stresses 3D object manipulation, and Reacher is continuous reaching.

For each environment, we train GC-IDM on frozen LeWM embeddings and evaluate it against the same pretrained LeWM backbone using its original default CEM planner configuration. We report results across two evaluation protocols ($n=50$ and $n=200$), a strict episode-level holdout, and present supplementary compute and ablation studies in the appendix.

5.2 Implementation Details

LeWM employs a ViT-based encoder and a transformer-based predictor, whereas GC-IDM is instantiated as a compact MLP with horizon conditioning. Comprehensive details regarding numerical hyperparameters, optimizer settings, and evaluation configurations are provided in Appendix F.

Compute accounting. GC-IDM incurs an additional one-time supervised fitting cost, though it relies on the same pretrained LeWM encoder and predictor shared with CEM. In our runs, this

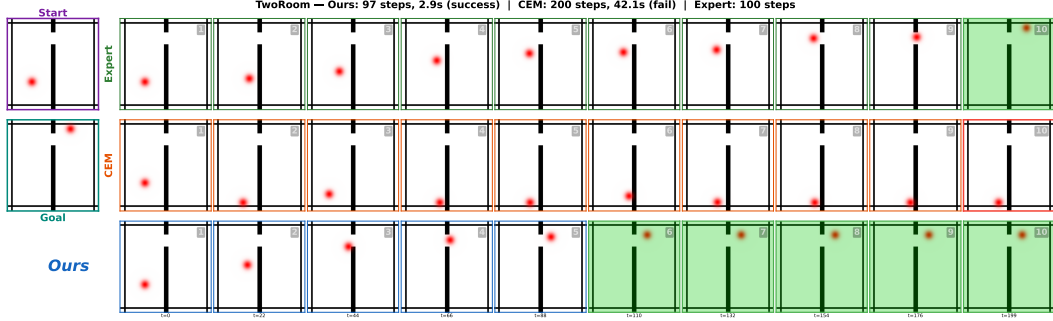


Figure 3: **Matched execution rollout on Two-Room.** Expert, CEM, and GC-IDM (ours) on the same episode from identical start and goal states. The first column shows the start (top) and goal (bottom); columns 1–10 are evenly spaced frames. Green shading marks success; red borders mark failure. CEM and GC-IDM share the same time axis; the expert row uses the dataset’s own time axis. GC-IDM reaches the goal in fewer steps with visibly smoother motion. On this environment, GC-IDM achieves 92% latent monotonicity (the agent’s latent distance to the goal decreases at every step in 92% of episodes) versus 73% for CEM, and $35.5\times$ lower action jerk (0.055 vs. 1.962).

extra training stage takes approximately 20 minutes on a single GPU per environment. Our primary comparison, therefore, contrasts amortized offline fitting with repeated online search. At inference time, GC-IDM executes one encoder pass and one MLP forward pass per control step, whereas CEM repeatedly invokes the latent predictor inside its optimization loop. All main experiments are run on an NVIDIA L4 GPU for Two-Room, Push-T, and Reacher, and on an NVIDIA A100 for OGBench-Cube (which requires more GPU memory for MuJoCo rendering and physics).

5.3 Main Results

Table 1 summarizes this comparative analysis. GC-IDM matches or outperforms CEM in seven of the eight environment–protocol cells while reducing the planning cost by $100\text{--}130\times$ per decision. The strongest gains occur on OGBench-Cube and Reacher, where GC-IDM is much faster and significantly more successful. The only exception is Push-T at $n=50$, where CEM is modestly better in mean success but remains more than two orders of magnitude slower in terms of planning time.

Two aspects of Table 1 stand out. First, the advantages are not merely computational: on OGBench-Cube and Reacher, GC-IDM is also substantially *more accurate*, challenging the assumption that CEM extracts essential long-horizon reasoning inaccessible to a local inverse model. Second, the one environment where CEM remains competitive, Push-T, is exactly the regime in which longer contact sequences make local inverse recovery inherently more challenging. This pattern supports our core hypothesis: inverse dynamics excels when the latent geometry ensures well-conditioned local action recovery, but its efficacy wanes when the task requires more delicate multi-step contact planning.

Three synergistic mechanisms drive this result (Figure 3 illustrates the effect on Two-Room). First, GC-IDM operates on real encoder outputs during both training and test time, eliminating the distribution shift that cripples pairwise IDMs (Appendix D). Second, re-encoding the current observation at every step delivers robust closed-loop correction: each single-step error is overwritten by a fresh observation rather than being integrated over a commit window. Third, even with a low pointwise R^2 , the IDM learns the average direction toward the goal, and per-step replanning accumulates these directionally correct predictions into successful trajectories. The ablations in Section 5.5 test each mechanism independently; Appendix A provides an analysis with error-propagation bounds.

5.4 Comparison with MPPI, iCEM, and Gradient-Based Planners

A compute sweep over 12 CEM configurations spanning a $500\times$ budget range shows that no configuration is simultaneously faster and more successful than GC-IDM on any environment (Figure B). To determine whether this gap extends beyond CEM’s specific sampling scheme, we evaluated all four environments using three alternative solvers from `stable_worldmodel` alongside CEM, each at the library’s default hyperparameters: MPPI (softmax-weighted path-integral sampling) [Williams

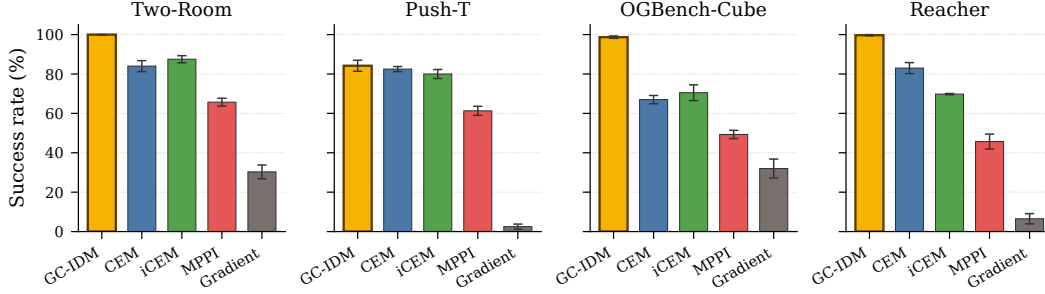


Figure 4: **Solver-family comparison, $n=200$, four environments.** Success rate and per-plan-call wall-clock, mean $\pm$ std over three training seeds. All sampling solvers use `stable_worldmodel` defaults; GradientSolver uses SGD with $1r=1.0$. GC-IDM is the highest-success method in every environment: it exceeds the best sampling baseline by 12.5 pp on Two-Room, 1.7 pp on Push-T, 28.2 pp on Cube, and 29.4 pp on Reacher, at 29–134 $\times$ lower per-plan-call cost. Appendix E.4 gives the $n=50$ protocol, per-episode timing, and full configuration details.

et al., 2017], iCEM (CEM with temporally correlated noise and elite reuse) [Pinneri et al., 2021], and GradientSolver (SGD backpropagation through the world-model predictor). All four sampling-based planners share the same frozen LeWM encoder, the same `PlanConfig`, and identical episode samples per (n, seed) cell. Figure 4 reports the $n=200$ summary; the full per-protocol table with $n=50$ rows and wall-clock timing is available in Appendix E.4.

At matched library defaults, GC-IDM remains the highest-success method in every (n, env) cell. The best sampling baseline differs by environment: iCEM on Two-Room (87.5%) and Cube (70.5%), and CEM on Push-T (82.5%) and Reacher (70.3%); however, none of them closes the performance gap. The first-order GradientSolver collapses across the board, reaching only 2.5–32.0% across the four environments; the default SGD step fails to recover trajectory-optimization-quality actions through the LeWM predictor within 30 iterations. Speedups hold across the entire family: GC-IDM is 103–134 $\times$ faster per plan call than iCEM, 29–34 $\times$ faster than MPPI, and 2.0–2.4 $\times$ faster than GradientSolver, further complementing the 104–130 $\times$ gap over CEM documented in Table 1.

5.5 Ablation Studies

Compute frontier. Sweeping CEM across 12 configurations (`num_samples` $\in \{30, 100, 300, 1000\}$, `n_steps` $\in \{2, 5, 10, 30\}$; 60–30,000 total rollouts per plan call), no single CEM configuration is simultaneously faster *and* more successful than GC-IDM on any environment: the upper-left win region is empty across the full 500 $\times$ compute sweep. On Push-T, the best CEM configuration (90% at 11.4 s per episode) exceeds GC-IDM’s 84.2% but at 35 $\times$ the planning cost. On the other three environments, CEM saturates below GC-IDM regardless of the allocated budget: Two-Room at 86%, OGBench-Cube at 68%, and Reacher at 76%. The full Pareto plots are provided in Figure B.

Goal-distance robustness. Table 2 demonstrates that GC-IDM maintains strong performance even as the start–goal offset increases. Two-Room performance is saturated at 100% across the full range, while Reacher and OGBench-Cube remain nearly flat. Push-T degrades more noticeably with distance, consistent with the view that longer contact-rich action chains are the primary regime where local inverse recovery becomes difficult.

Horizon supervision. Table 3 shows that integrating multi-step horizon supervision is necessary, especially on Push-T. Very short horizons impair performance, while gains largely saturate by $H_{\max}=50$ at the 50-step evaluation budget.

5.6 Trajectory Quality: Smoothness and Directness

Beyond evaluating binary success rates, GC-IDM also produces qualitatively better trajectories than CEM. Two metrics capture complementary aspects of trajectory quality: *action smoothness* (jerk) and *directness of approach* in latent space (monotonicity).

Table 2: Ablation study on goal-distance.

Goal offset (steps)	5	10	15	25	35	50
Two-Room	100.0	100.0	100.0	100.0	100.0	100.0
Reacher	99.5	100.0	99.5	99.5	99.0	99.0
Push-T	94.0	90.5	88.5	84.5	76.0	70.0
OGBench-Cube	98.5	99.0	99.0	99.0	95.0	93.0

Table 3: Ablation study on $H_{\max}$.

$H_{\max}$	5	10	25	50	100
Two-Room	81.5	99.0	100.0	100.0	100.0
Push-T	46.5	67.5	84.0	85.0	83.5
Reacher	88.0	97.0	99.5	99.5	99.5
OGBench-Cube	69.5	79.5	99.0	99.0	98.0

Table 4: **Trajectory quality: action jerk and latent monotonicity.** Both methods evaluated on the same 50 episodes per environment (seed 42, goal offset 25, budget 50) from matched initial states and goals. Jerk is the mean ℓ_2 norm of the second finite difference of the action sequence (lower is smoother). Monotonicity is the fraction of episodes in which latent distance to the goal decreases at every step (higher is more direct).

Environment	Action Jerk $\downarrow$			Monotonicity $\uparrow$	
	GC-IDM	CEM	Ratio	GC-IDM	CEM
Two-Room	0.055	1.962	$35.5\times$	92%	73%
Reacher	0.043	1.103	$25.7\times$	80%	63%
OGB-Cube	0.083	1.721	$20.7\times$	69%	71%
Push-T	0.028	0.429	$15.3\times$	100%	71%

Metrics. *Action jerk* is the mean second finite difference of the action sequence: $\frac{1}{T-2} \sum_{t=2}^T \|a_t - 2a_{t-1} + a_{t-2}\|_2$. Lower jerk corresponds to smoother, more physically plausible control. Whereas CEM replans in open-loop 5-step blocks, creating discontinuities at block boundaries, GC-IDM acts closed-loop at every step with actions inherited from expert demonstrations via the IDM training set. *Latent monotonicity* measures $\|z_t - z_{\text{goal}}\|_2$ at every step and reports the fraction of episodes in which this distance decreases monotonically, meaning the agent moves steadily toward the goal in latent space without backtracking. See supplementary videos for rollout comparisons across environments.

Table 4 reports both metrics on matched episodes. GC-IDM produces $15\text{--}36\times$ smoother actions across all four environments, and approaches the goal more directly in latent space on three of the four tasks (Cube results in a near-tie at 69% vs. 71%). The smoothness advantage is structural: closed-loop single-step imitation inherits the continuity of expert-demonstrations, while CEM’s open-loop replanning injects block-boundary discontinuities every 25 raw environment steps. The monotonicity advantage is a direct consequence of re-encoding at every step: each action is chosen from the true current latent state, avoiding the drift that accumulates when CEM commits to an open-loop block from a predicted latent trajectory.

6 Conclusion and Future Work

We propose GC-IDM, a learned planning method that eliminates the need for iterative test-time search in pretrained JEPA world models. While LeWM is already an efficient world model, featuring $\sim 15\text{M}$ parameters, trainable on a single GPU in a few hours, and planning $48\times$ faster than foundation-model-based alternatives [Maes et al., 2026], its CEM planner still dominates the overall inference cost. GC-IDM removes this bottleneck: a 1.5M-parameter MLP trained in ~ 20 minutes per environment on frozen LeWM embeddings replaces the entire 9,000-rollout CEM loop with a single forward pass per step. This approach matches or exceeds CEM’s success rate in seven of the eight environment-protocol cells while reducing planning cost by $100\text{--}130\times$.

Crucially, this performance advantage extends to the broader family of test-time planners: MPPI, iCEM, and gradient-based optimization all fall below GC-IDM in terms of success rate despite incurring substantially higher computational costs. These results support a simple and direct interpretation: when a world model’s latent space is sufficiently well structured by regularization, the planning problem can be shifted away from online optimization and completely into learned inference. An important direction for future work is to test how far this regime extends to settings with stronger irreversibility, longer-horizon contact structures, higher-dimensional action spaces, and different latent architectures. Additionally, we aim to explore the role of the regularizer itself by comparing isotropic and non-isotropic latent spaces on the same environments.

References

- Pulkit Agrawal, Ashvin V Nair, Pieter Abbeel, Jitendra Malik, and Sergey Levine. Learning to poke by poking: Experiential learning of intuitive physics. In *NeurIPS*, 2016.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video PreTraining (VPT): Learning to act by watching unlabeled online videos. *NeurIPS*, 2022.
- Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations*, 2021.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. *International Conference on Machine Learning*, 2022.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. *International Conference on Learning Representations*, 2024.
- Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *International Journal of Robotics Research*, 5(1):90–98, 1986.
- Yunhao Luo and Yilun Du. Grounding video models to actions through goal-conditioned exploration. In *International Conference on Learning Representations (ICLR)*, 2025. *arXiv preprint arXiv:2411.07223*.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- Sacha Morin, Moonsub Byeon, Alexia Jolicoeur-Martineau, and Sébastien Lachapelle. On the sample efficiency of inverse dynamics models for semi-supervised imitation learning. *arXiv preprint arXiv:2602.02762*, 2026.
- Jonas Pai, Liam Achenbach, Victoriano Montesinos, Benedek Forrai, Oier Mees, and Elvis Nava. mimic-video: Video-action models for generalizable robot control beyond VLAs. *arXiv preprint arXiv:2512.15692*, 2025.
- Keiran Paster, Sheila A. McIlraith, and Jimmy Ba. Planning from pixels using inverse dynamics models. In *International Conference on Learning Representations (ICLR)*, 2021.
- Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning*, 2017.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. *IEEE/CVF International Conference on Computer Vision*, 2023.

- Cristina Pinneri, Shambhuraj Sawant, Sebastian Blaes, Jan Achterhold, Joerg Stueckler, Michal Rolinek, and Georg Martius. Sample-efficient cross-entropy method for real-time planning. In Jens Kober, Fabio Ramos, and Claire Tomlin, editors, *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 1049–1065. PMLR, 16–18 Nov 2021. URL <https://proceedings.mlr.press/v155/pinneri21a.html>.
- Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.
- Lukas Schäfer, Pallavi Choudhury, Abdelhak Lemkhenter, Chris Lovett, Somjit Nath, Luis França, Matheus Ribeiro Furtado de Mendonça, Alex Lamb, Riashat Islam, Siddhartha Sen, John Langford, Katja Hofmann, and Sergio Valcarcel Macua. When does predictive inverse dynamics outperform behavior cloning? *arXiv preprint arXiv:2601.21718*, 2026.
- Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features. *arXiv preprint arXiv:2211.10831*, 2022.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: a case for planning with latent dynamics models. In *WRL@ICLR 2025*, 2025.
- Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.15109.
- Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic mpc for model-based reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017. doi: 10.1109/ICRA.2017.7989202.
- Amber Xie, Oleh Rybkin, Dorsa Sadigh, and Chelsea Finn. Latent diffusion planning for imitation learning. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2504.16925.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

A Why GC-IDM Works: Mechanism Analysis

Here we develop each in detail and provide the error-propagation analysis that underpins the closed-loop argument.

Error propagation and replan frequency. Both CEM and GC-IDM are model predictive controllers with a receding horizon, and both are therefore closed-loop in the MPC sense. What differs is the interval between re-encodings. GC-IDM re-encodes the current observation and re-computes the action every environment step; LeWM’s default CEM configuration (`receding_horizon=5`, `action_block=5`) commits 25 raw environment actions before re-encoding, so a 50-step episode contains 50 re-encodings for GC-IDM and 2 for CEM.

Let $\epsilon_t = \hat{\mathbf{a}}_t - \mathbf{a}_t^*$ denote the single-step action error at time t , where $\mathbf{a}_t^*$ is the optimal action toward the goal, and let δ_{t+1} denote the resulting observation divergence. Inside any window of T environment steps during which the controller commits actions without re-encoding, divergence accumulates:

$$\delta_T^{\text{open}} \leq L \sum_{t=1}^T \|\epsilon_t\|, \quad (8)$$

where L is the Lipschitz constant of the environment transition with respect to actions. A controller that re-encodes after every step and recomputes $\hat{\mathbf{a}}_{t+1}$ from the fresh observation satisfies the tighter per-step bound

$$\delta_{t+1}^{\text{closed}} \leq L \cdot \|\epsilon_t\|, \quad (9)$$

because each step’s error is overwritten by the next re-encoding rather than integrated forward over additional committed actions. GC-IDM operates under (9) at every step; CEM operates under (8) with $T=25$ inside each commit window and under (9) across window boundaries.

Three mechanisms in detail. GC-IDM achieves 100% success on Two-Room despite a training R^2 of only 0.20. Three mechanisms account for this:

Mechanism 1: No train/test distribution shift. GC-IDM operates on real ViT-encoded observations at both training and test time, eliminating the distribution shift that forces the pairwise IDM (Appendix D) into extreme noise augmentation ($\sigma=1.5$). This is confirmed by the noise ablation (Table E): every environment prefers $\sigma=0$.

Mechanism 2: Closed-loop correction via re-encoding. Re-encoding the current observation at every environment step delivers the error bound in Equation (9): any single-step prediction error is overwritten one step later by a fresh observation. This mechanism predicts that GC-IDM should be insensitive to noise injection during training, because at test time it sees clean encoder outputs anyway, exactly what Table E shows.

Mechanism 3: Directionally correct predictions accumulate. Even at low R^2 the IDM learns the average direction toward the goal at each state, and per-step replanning accumulates these noisy but directionally correct predictions into successful trajectories, similar in spirit to potential-field navigation [Khatib, 1986]. The latent monotonicity measurements in Table 4 provide direct evidence: on Two-Room, 92% of GC-IDM episodes show monotonically decreasing latent distance to the goal at every step. The same effect appears in the bias–variance decomposition of Schäfer et al. [2026]: future-state conditioning trades a small predictor bias for a large variance reduction in the action estimator, and a low- R^2 pointwise estimator can therefore induce a high-quality closed-loop policy.

Together these three mechanisms replace search over action sequences with a directed step in one forward pass: at each environment step, GC-IDM commits to the action that moves the latent state toward the goal under its current estimate, the environment evolves, and a fresh encoding replaces the estimate for the next step. No candidate trajectories are ever enumerated or scored. Each mechanism implies a testable empirical prediction: closed-loop re-encoding $\rightarrow$ noise insensitivity (Table E); horizon supervision $\rightarrow$ removal collapses performance even at higher capacity (Table F); smooth latent metric $\rightarrow$ no benefit from extra capacity (Table H).

B Theoretical Analysis: Isotropy and Inverse-Dynamics Conditioning

Under idealized conditions on the latent geometry, the inverse-dynamics problem has a favorable first-order form. This analysis is *conditional*: it identifies geometric properties that would make inverse dynamics easier to learn, but we have not verified that LeWM’s latent space satisfies these conditions in full, nor have we ablated against a non-isotropic baseline. The empirical results in the main paper stand independently of this theoretical framing.

Let s_t denote the underlying state, a_t the action, and $z_t = e(s_t)$ the latent embedding produced by the frozen encoder. Consider the one-step latent transition map

$$F(z_t, a_t) \triangleq e(f(s_t, a_t)), \quad (10)$$

where f is the environment dynamics and $s_t = e^{-1}(z_t)$ is understood only locally on the data manifold.

Assumption 1 (Isotropic latent statistics). On the demonstration distribution the latent variable is approximately whitened and isotropic in the second-order sense,

$$\mathbb{E}[z_t] = 0, \quad \text{Cov}(z_t) = I, \quad (11)$$

with the additional modeling idealization that the local latent density is close to Gaussian on the region visited by demonstrations.

Assumption 2 (Local regularity and action identifiability). Near the demonstration manifold: (i) e is differentiable and locally bi-Lipschitz; (ii) f is differentiable in (s, a) ; and (iii) for fixed z_t , the Jacobian $\partial F(z_t, a)/\partial a$ has full column rank.

Under these assumptions, linearizing at (s_t, a_t) gives, for a nearby action perturbation δa ,

$$F(z_t, a_t + \delta a) \approx F(z_t, a_t) + B_t \delta a, \quad B_t \triangleq \frac{\partial F(z_t, a_t)}{\partial a}. \quad (12)$$

Defining the induced latent displacement $\delta z \triangleq F(z_t, a_t + \delta a) - F(z_t, a_t)$, the local inverse problem becomes $\delta z \approx B_t \delta a$. When B_t has full column rank, the minimum-norm local solution is

$$\delta a \approx B_t^\dagger \delta z = (B_t^\top B_t)^{-1} B_t^\top \delta z, \quad (13)$$

where $B_t^\dagger$ is the Moore–Penrose pseudoinverse.

Proposition 1 (Local inverse map). *Under Assumption 2, the implicit function theorem yields a local inverse map G satisfying*

$$a_t = G(z_t, z_{t+1}), \quad z_{t+1} = F(z_t, G(z_t, z_{t+1})), \quad (14)$$

and this inverse is continuous (indeed differentiable) in a neighborhood of each training point.

In words, as long as actions induce locally distinguishable motion in latent space, inverse dynamics is a well-posed local regression problem.

Why isotropy would improve conditioning. The learnability question is controlled by the singular spectrum of $B_t = \partial F/\partial a$. By the chain rule,

$$B_t = J_e(s_{t+1}) J_f^{(a)}(s_t, a_t), \quad (15)$$

where J_e is the encoder Jacobian and $J_f^{(a)}$ is the dynamics Jacobian with respect to action. If, in addition, the encoder is locally near-isometric in the sense that

$$mI \preceq J_e(s)^\top J_e(s) \preceq MI \quad \text{with } M/m \text{ small}, \quad (16)$$

then the singular values of $J_e(s)$ all lie in $[\sqrt{m}, \sqrt{M}]$, so

$$\kappa(B_t) \leq \kappa(J_e(s_{t+1})) \kappa(J_f^{(a)}(s_t, a_t)) \leq \sqrt{M/m} \kappa(J_f^{(a)}(s_t, a_t)). \quad (17)$$

Thus, when the encoder is close to isotropic at the Jacobian level, it does not introduce substantial additional ill-conditioning beyond that already present in the action-to-state map. SIGReg fits this picture by discouraging collapsed or disproportionately amplified latent directions. The rigorous claim is conditional: distributional isotropy alone does not imply Equation (16), but if LeJEPa-style isotropic statistics are accompanied by local near-isometry of the encoder, then the induced inverse problem satisfies Equation (17) and is easier to approximate with a small model.

C More Qualitative Comparisons

Comparison on latent planning trajectories. Figure A provides a qualitative view of GC-IDM and CEM rollouts across representative episodes. On Two-Room and Reacher, both methods produce visually coherent trajectories, but GC-IDM’s per-step action sequences are smoother because there is no re-optimization jitter from the CEM sampling loop. On Push-T and OGBench-Cube, the contact-rich phases reveal the main behavioral difference: GC-IDM commits to a single directed step per environment step and relies on re-encoding to correct, while CEM’s action sequences show larger step-to-step variance from the repeated sampling-and-refinement cycle. When GC-IDM fails on Push-T, it typically undershoots the rotation component of the goal pose rather than diverging entirely, consistent with the view that longer contact sequences are the regime where local inverse recovery becomes harder.

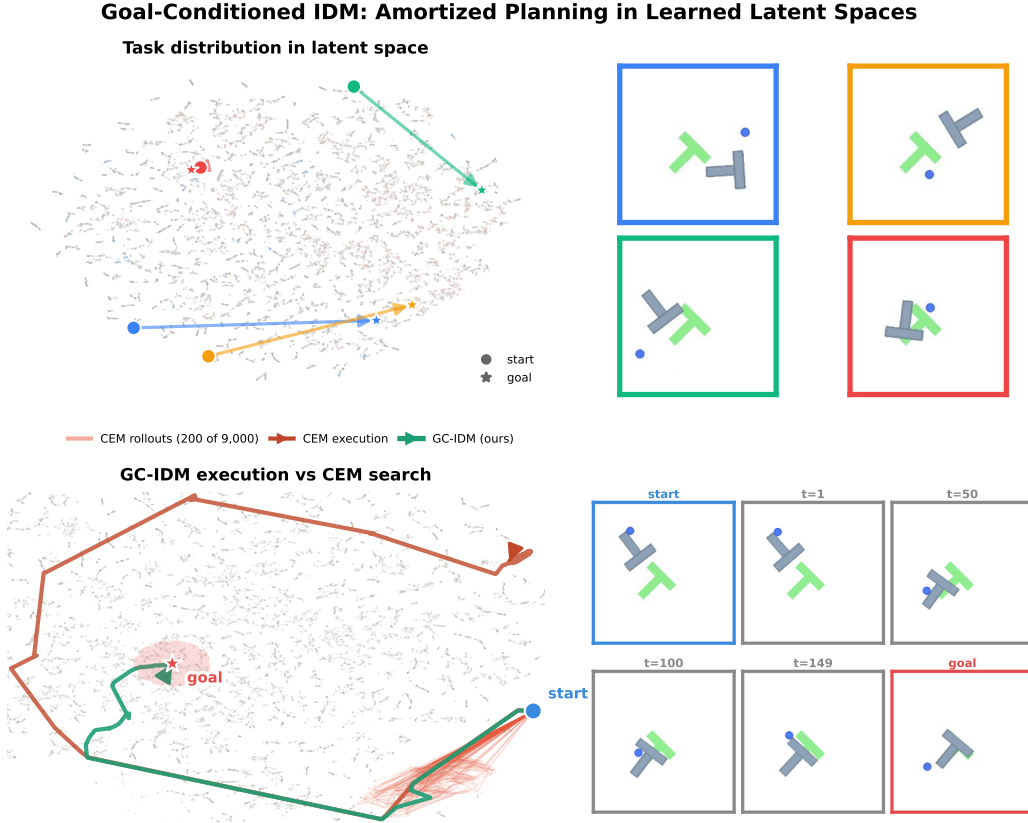


Figure A: **Comparisons on latent trajectory rollouts.** *Top:* Four goal-conditioned tasks in the PushT environment, visualized in the 192-dimensional LeWM latent space (t-SNE projection; grey points are 20K training embeddings). Each colored circle marks a different starting configuration (right panels: the grey T-block and blue agent vary across tasks), while stars mark the corresponding goal observations. All four tasks share the same physical target (the green T-block at a fixed position), but goal embeddings differ because the encoder represents the full scene, including agent and block positions at the goal timestep, not the target alone. *Bottom:* Execution comparison on a single task. Light red lines visualize 200 of the 9,000 random candidate rollouts evaluated per CEM plan call, each spanning a 5-step prediction horizon; CEM replans every 25 environment steps. The dark red path shows CEM’s actual executed trajectory; the green path shows GC-IDM (ours). GC-IDM produces one action per step via a single forward pass through the inverse dynamics model, requiring no iterative search. Right panels show the GC-IDM execution progression from start to goal.

CEM pareto frontier analysis. As is shown in Fig.B, each panel sweeps CEM across 12 configurations (`num_samples` $\in$ {30, 100, 300, 1000}, `n_steps` $\in$ {2, 5, 10, 30}; 60–30,000 total rollouts), evaluated on 100 in-distribution episodes per configuration at seed 42. Dot color encodes refinement

iterations and dot size encodes num_samples. The dashed grey curve is the CEM upper-left Pareto envelope. The shaded yellow region marks configurations strictly faster and more successful than GC-IDM (gold star); it is empty in every panel across the full $500 \times$ compute sweep.

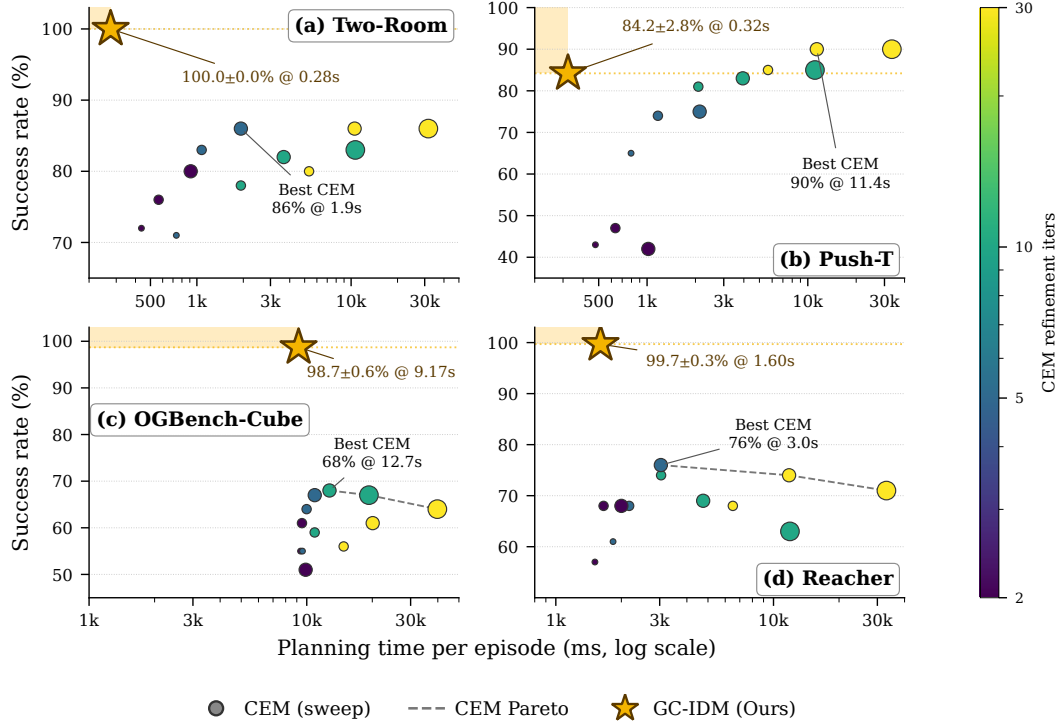


Figure B: CEM compute vs. success rate across four LeWM environments.

Comparisons on execution rollout frames. Each figure below shows four matched episodes per environment: expert, CEM, and GC-IDM (ours) from identical start and goal states. The first column shows the start (top) and goal (bottom); columns 1–10 are evenly spaced frames. Green shading marks success; red borders mark failure. CEM and GC-IDM share the same time axis; the expert row uses the dataset’s own time axis. Evaluation budget is 200 steps; goal offset is 100 for Two-Room and 200 for the remaining three. Results are organized by environment: Two-Room (Figure C), Push-T (Figure D), OGBench-Cube (Figure E), and Reacher (Figure F).

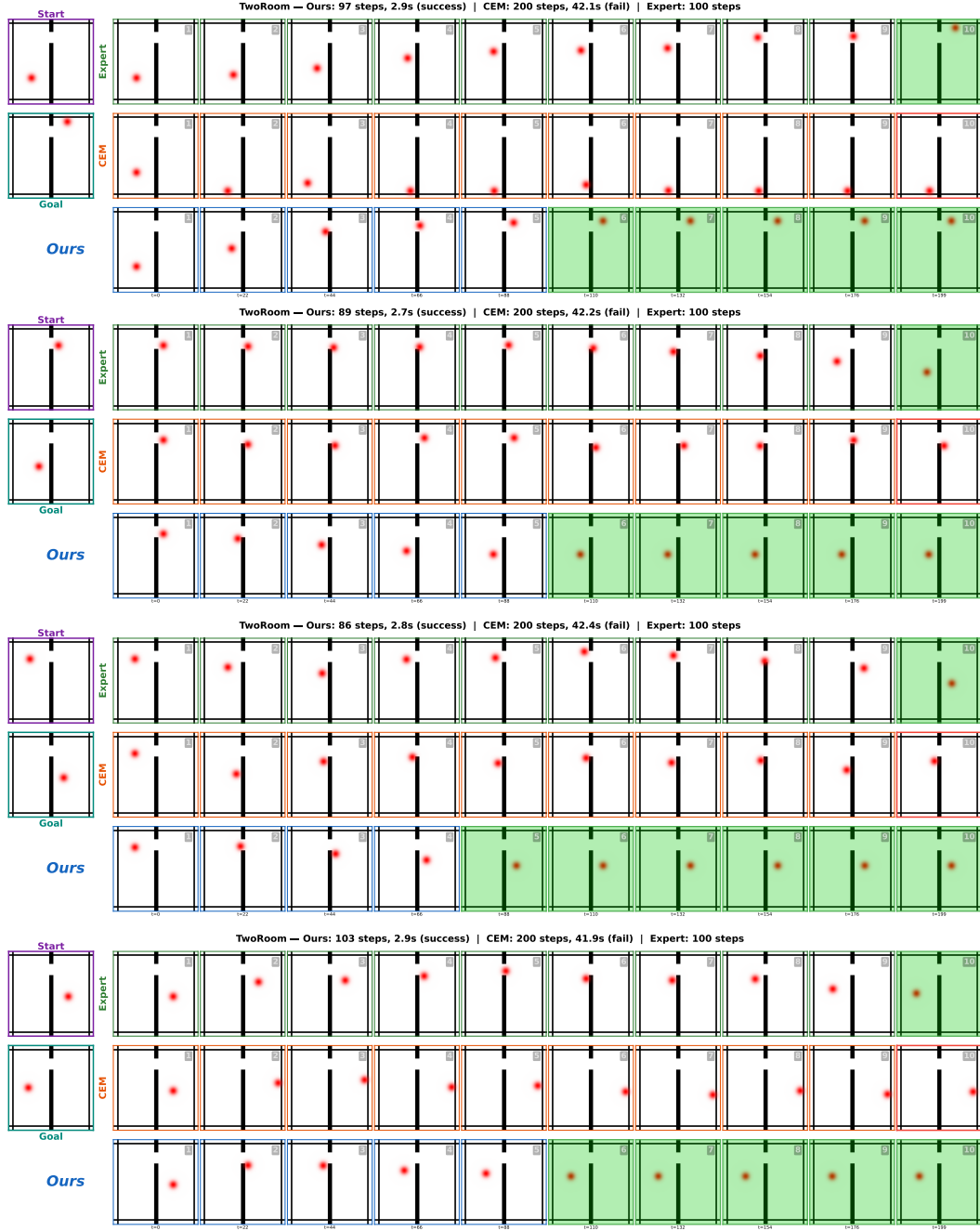


Figure C: **Two-Room: matched execution rollouts (4 episodes).**

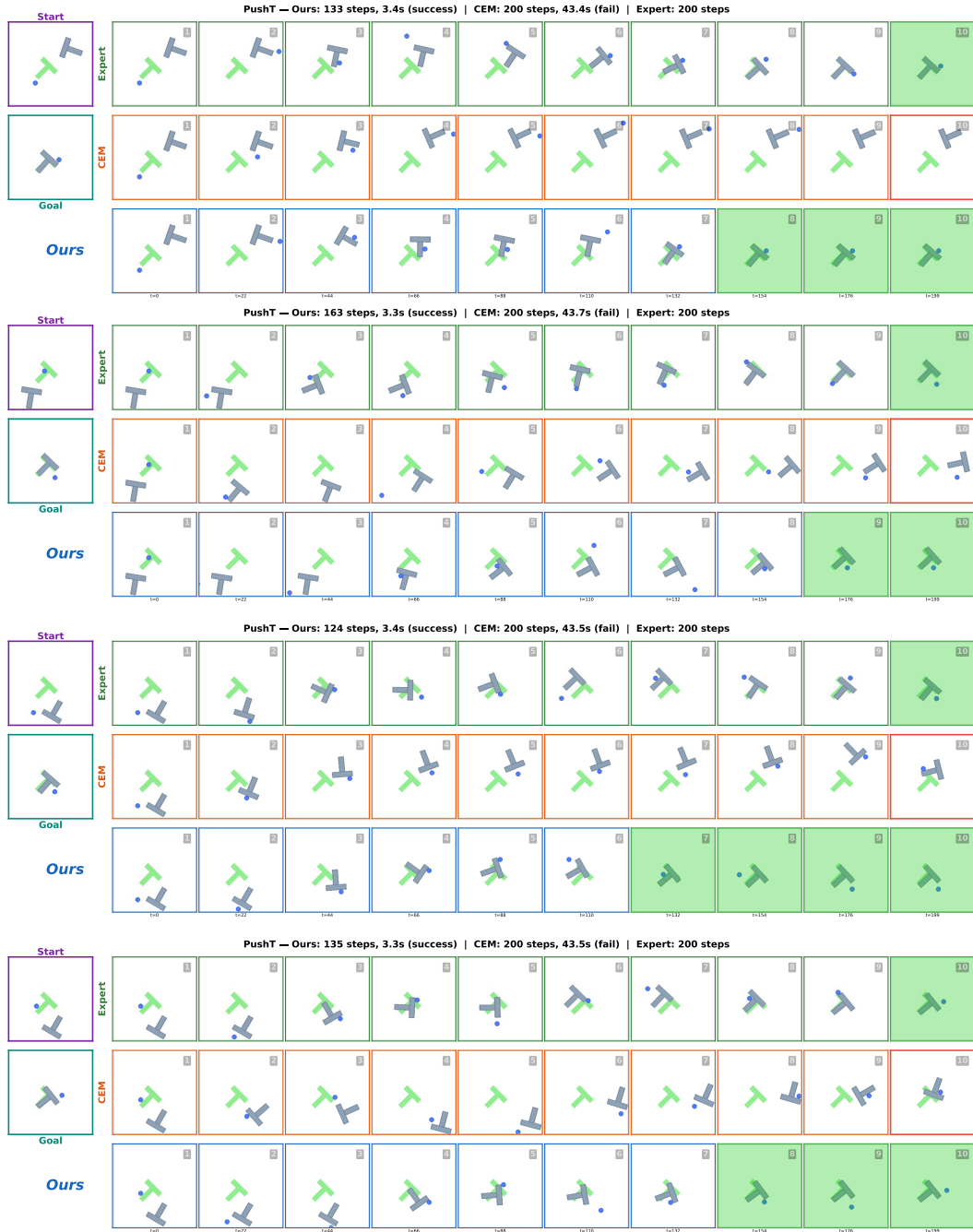


Figure D: Push-T: matched execution rollouts (4 episodes).

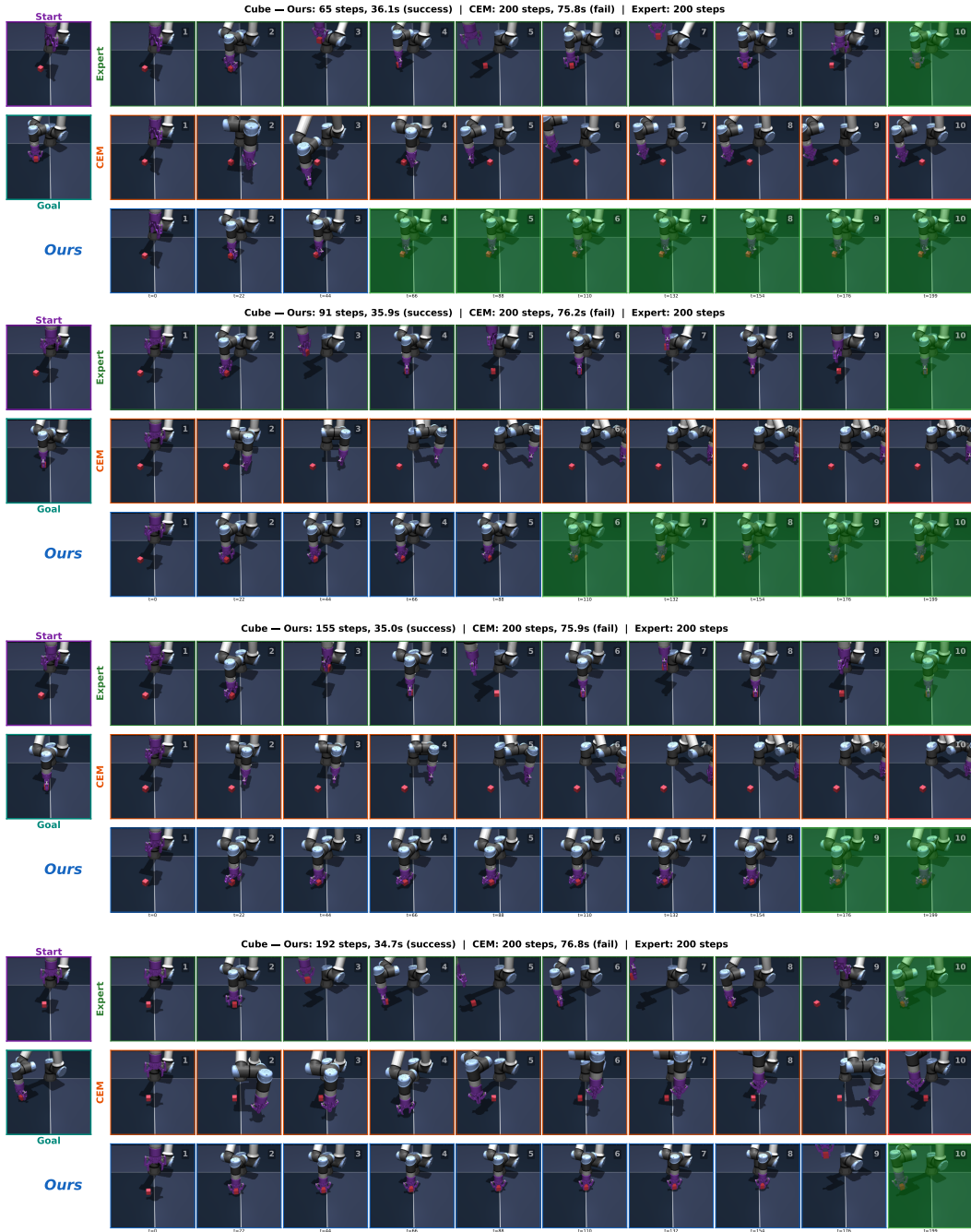


Figure E: OGBench-Cube: matched execution rollouts (4 episodes).

D Pilot Study on Pairwise IDM

The pairwise IDM motivates the goal-conditioned formulation. Here we give the full pairwise method, the oracle experiment that was the original empirical anchor, the noise sweep, and the closed-loop failure analysis.

D.1 Pairwise IDM

The pairwise IDM learns the mapping $(z_t, z_{t+1}) \rightarrow a_t$ from consecutive latent state pairs. Given a trained LeWM encoder, we extract embeddings for all frames in the training dataset and construct supervision triples (z_t, z_{t+1}, a_t) where a_t is the ground-truth action that produced the transition.

Architecture. The pairwise IDM is a 3-layer MLP with LayerNorm and GELU activations:

$$\hat{a}_t = \text{idm}_\psi(z_t \parallel z_{t+1}), \quad (18)$$

where $\parallel$ denotes concatenation. With $d=192$ (LeWM embedding dimension), hidden dimension 512, and action dimension 2, the model has $\sim 730K$ parameters.

Frameskip. LeWM trains with frameskip $k=5$, meaning the predictor operates on every 5th frame. A natural choice is to train the IDM at the same temporal resolution, predicting concatenated action chunks $[a_t, \dots, a_{t+k-1}]$ from pairs (z_t, z_{t+k}) . However, this mapping is fundamentally ambiguous: many distinct k -step action sequences can connect the same two latent states. We find that frameskip-5 IDMs achieve $R^2 \approx 0$ on Two-Room. Training with frameskip-1 (single raw actions from consecutive frames) yields $R^2 = 0.993$, and we adopt this throughout.

D.2 Noisy Conditioning

A key insight from mimic-video [Pai et al., 2025] is that the IDM must be robust to imperfect conditioning signals at test time. During training, the IDM receives ground-truth encoder embeddings; at test time it receives interpolated or predictor-generated embeddings, which differ in distribution. To bridge this gap, we add Gaussian noise to the input embeddings during training:

$$z_{\text{noisy}} = z + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, I). \quad (19)$$

D.3 Oracle Experiment and Noise Ablation

The pairwise IDM receives ground-truth encoder embeddings during training but interpolated or predictor-generated embeddings at test time. Following mimic-video [Pai et al., 2025], we add Gaussian noise $z_{\text{noisy}} = z + \sigma \cdot \epsilon$, $\epsilon \sim \mathcal{N}(\mathbf{0}, I)$ to bridge this distribution gap. Two experiments characterize its effect on Two-Room. Table A tests a single IDM trained at $\sigma=0$ against increasing test-time noise on ground-truth embedding pairs, with no environment interaction. At $\sigma=0$ reconstruction is near-perfect ($R^2=0.993$), confirming that LeWM’s latent transitions encode essentially all the information needed to determine the action; reconstruction degrades gracefully up to $\sigma \approx 0.5$ and collapses beyond $\sigma=1$. Table B trains eight separate IDMs at different σ values and evaluates each closed-loop with lerp planning. The optimal training noise is $\sigma=1.5$ (34% success), corresponding to 150% of the per-dimension embedding standard deviation, reflecting the large distribution shift between ground-truth and linearly interpolated embeddings. The gap between oracle quality ($R^2 = 0.993$) and the best closed-loop success (34%) confirms that the bottleneck lies in trajectory generation, not action decoding.

D.4 Closed-Loop Performance and Failure Mode

Given an encoded start z_{start} and goal z_{goal} , we generate a latent trajectory via linear interpolation $z_i = (1-\alpha_i)z_{\text{start}} + \alpha_i z_{\text{goal}}$ for $\alpha_i = i/H$, then decode actions from consecutive pairs using the pairwise IDM. Optionally, a refinement step rolls out the decoded actions through the predictor to obtain a corrected trajectory, which is then re-decoded; this can be iterated K times. Table C reports the resulting performance on Two-Room.

Table A: Oracle noise robustness: single $\sigma=0$ model, test-time noise on ground-truth pairs.

Test σ	MSE	Cos Sim	R^2
0.00	0.005017	0.9966	0.9933
0.01	0.005032	0.9966	0.9933
0.02	0.005074	0.9966	0.9933
0.05	0.005733	0.9961	0.9924
0.10	0.008802	0.9936	0.9883
0.20	0.018036	0.9860	0.9761
0.50	0.081068	0.9416	0.8924
1.00	0.321688	0.7303	0.5731
2.00	0.720835	0.3286	0.0435

Table B: Training noise ablation: 8 separate models, closed-loop with lerp.

Train σ	Val MSE	Success
0.0	0.005540	18%
0.1	0.005986	20%
0.2	0.006591	26%
0.5	0.009219	24%
0.8	0.011906	28%
1.0	0.013480	26%
1.5	0.019367	34%
2.0	0.028369	28%

Table C: Pairwise IDM planning results. Linear interpolation (lerp) is the fundamental bottleneck: even with predictor-based refinement and multiple candidates, success rates plateau well below CEM.

Environment	Configuration	Success	ms/plan	Speedup
Two-Room	Lerp only ($\sigma=1.5$)	34%	64	16×
	+ Refine×1	36%	119	8×
	+ Refine×3	42%	285	3.5×
	+ Refine×3, 10 candidates	48%	195	5×
	CEM (baseline)	87%	1000	1×

Linear interpolation fails on complex tasks. The pairwise IDM with linear interpolation achieves at most 48% on Two-Room (vs. CEM’s 87%). The failure mode is clear: linear interpolation assumes the optimal latent path between start and goal is a straight line. For Two-Room, this assumption fails because the path must curve through the doorway.

Refinement helps but saturates. Predictor-based refinement (rolling out decoded actions through the predictor to correct the trajectory) improves success from 34% to 42% on Two-Room. Additional iterations beyond 3 yield diminishing returns as predictor errors compound. Adding 10 noisy trajectory candidates with selection yields a further 6% improvement to 48%, but the overall gap to CEM (87%) remains large.

E More Ablation Study

The main text reports ablations on goal distance (Table 2) and $H_{\max}$ (Table 3). Here we report the remaining ablations: data efficiency, noise injection, horizon-embedding ablation, architecture sensitivity, evaluation budget, noise-schedule comparison, and the CEM compute-sweep data.

E.1 Data Efficiency

GC-IDM’s data requirements vary sharply by environment, with three of the four tasks approaching saturation well below the full training set. We retrain GC-IDM on fractions of the training data, subsampled at the episode level, and evaluate with the same protocol as Table 1. Table D records the data efficiency ablation results. Two-Room and Reacher are near-saturated with as little as 5% of the data. OGBench-Cube climbs smoothly to near-saturation by full data. Push-T shows no plateau: success rate climbs monotonically, so additional demonstrations would likely still improve GC-IDM on that environment.

Table D: **Data efficiency of GC-IDM.** Success rate (%) across all four environments versus training data fraction (subsampled at the episode level, fixed seed).

Training data fraction	1%	5%	10%	25%	50%	100%
Two-Room	87.0	100.0	100.0	100.0	100.0	100.0
Push-T	40.5	58.5	67.0	76.5	78.0	85.0
OGB-Cube	60.5	80.0	86.0	92.5	98.5	99.0
Reacher	38.5	99.5	99.5	99.5	99.5	99.5

E.2 Noise Augmentation is Unnecessary for GC-IDM

GC-IDM’s optimal noise level is $\sigma=0$ on every environment, in contrast to the pairwise IDM, which requires $\sigma=1.5$ to be usable. Because GC-IDM operates on real encoder outputs at both training and test time, the distribution gap is zero by construction. Table E confirms this: on Push-T, increasing σ from 0 to 0.5 drops success rate from 85.5% to 67.0%. No environment prefers $\sigma > 0$.

Table E: **Noise ablation for GC-IDM.** Fixed- σ Gaussian noise injected on input embeddings during training only. Unlike the pairwise IDM which requires $\sigma=1.5$, GC-IDM degrades monotonically with noise on Push-T (and mildly on OGBench-Cube) and is invariant on Two-Room and Reacher. There is no distribution gap to bridge.

σ	0.0	0.01	0.05	0.1	0.2	0.5
Two-Room	100	100	100	100	100	100
Push-T	85.5	81.5	81.0	81.0	78.0	67.0
OGB-Cube	99.0	99.0	99.0	98.5	98.5	96.0
Reacher	99.5	99.5	99.5	99.5	99.5	99.5

E.3 Role of the Horizon Embedding

The horizon embedding h_t is the only input to GC-IDM that a standard goal-conditioned behavior-cloning policy does not have. If the horizon embedding is load-bearing, removing it should collapse performance, and that collapse should not be recoverable by scaling width or depth. Table F confirms this: removing horizon supervision entirely ($H_{\max}=1$) reduces success by an average of 42 percentage points across the four environments, and widening the network to 1024 hidden units and 4 layers while keeping $H_{\max}=1$ does not recover the gap. Capacity is not the issue; the horizon embedding supplies supervision that extra width cannot substitute for.

Table F: **Horizon-embedding ablation.** Success rate (%) at seed 42, $n=200$ in-distribution. $H_{\max}=1$ is equivalent to single-step goal-conditioned BC in latent space and fails because the model is never trained on distant goals; even a $2\times$ -wider backbone cannot recover the lost supervision signal.

Variant	Two-Room	Push-T	Cube	Reacher
$H_{\max}=1$ (next-frame only)	34.0	35.0	62.5	82.5
$H_{\max}=5$	81.5	46.5	69.5	88.0
$H_{\max}=1$, hidden=1024, 4 layers	28.5	31.5	60.5	77.5
$H_{\max}=50$ (ours)	100.0	85.0	98.7	99.5

E.4 Full Solver-Family Comparison

This section reports the full configuration details, the $n=50$ LeWM-parity protocol, and per-episode wall-clock timing in Table G.

Solver configurations. All four baselines use `stable_worldmodel`’s default hyperparameters. The sampling solvers (CEM, MPPI, iCEM) share `num_samples=300`, `n_steps=30`, `topk=30`, `var_scale=1.0`, giving 9,000 rollouts per plan call. MPPI additionally uses `temperature=0.5` (library default); iCEM additionally uses colored-noise `noise_beta=2.0`, keep-fraction `alpha=0.1`, `n_elite_keep=5`, and mean-return at the final iteration. The first-order GradientSolver uses SGD with `lr=1.0`, `action_noise=0`, `var_scale=1.0`, `n_steps=30`, and `num_samples=2`. All solvers share `PlanConfig(horizon=5, receding_horizon=5, action_block=5)`.

Table G reports the full comparison across both sample sizes. GC-IDM is the highest-success method in all eight (n, env) cells except Push-T at $n=50$, where CEM (89.3 ± 6.4) sits above GC-IDM (84.7 ± 5.0) within overlapping seed noise; at $n=200$ the ranking reverses. Among the sampling baselines, iCEM is the strongest alternative on Two-Room and OGBench-Cube, while CEM leads on Push-T and Reacher—but none closes the gap to GC-IDM. GradientSolver underperforms all other methods on every environment, reaching at most 32.0% on OGBench-Cube, indicating that first-order backpropagation through the LeWM predictor does not produce competitive action sequences at the default step size.

E.5 Architecture Hyperparameter Ablation

We sweep hidden dimension (128, 256, 512, 1024) and depth (1–5 layers) on all four environments at fixed $H_{\max}=50$ and $\sigma=0$. Table H reports the results. Two-Room and Reacher are fully saturated across every configuration, confirming that these environments are solved by the control structure rather than model capacity. Push-T is the informative environment: hidden dimension has negligible effect (81–84% across a $8\times$ width range), whereas depth shows a monotone trend from 70.5% at 1 layer to 87.5% at 5 layers, suggesting that representational depth matters more than width for contact-rich tasks. The default 3-layer configuration (85.0%) sits within 2.5 pp of the best observed setting and was selected for compute efficiency.

E.6 Noise Schedule

The main appendix reports that fixed- σ injection degrades GC-IDM monotonically. We additionally tested a *uniform* schedule that samples σ per-batch from $[0, \sigma_{\max}]$, in case time-varying noise provides robustness without the saturation cost of fixed noise. Table I reports the results. The uniform schedule does not recover any of the performance lost to noise injection: Push-T sits in a narrow 78–80.5% band regardless of schedule, which is 4–7 pp below the no-noise baseline (85.5%). Two-Room, Reacher, and OGBench-Cube are invariant to both schedule and magnitude. The conclusion that GC-IDM does not benefit from noise augmentation is robust to schedule choice.

E.7 Evaluation Budget and Long-Horizon Control

The default evaluation budget is 50 environment steps. We examine how quickly GC-IDM reaches its asymptotic success rate as the budget increases, and whether performance degrades at budgets far beyond training-time horizons. Table J sweeps the budget from 5 to 100 steps on all four environments.

Table G: **Full solver-family comparison across four environments and two sample sizes.** Success rate and timing are mean over three training seeds (42, 123, 456). GC-IDM is the highest-success method in all eight (n, env) cells except Push-T at $n=50$, where CEM (89.3 ± 6.4) sits above GC-IDM (84.7 ± 5.0) within overlapping seed noise; at $n=200$ the ranking reverses.

Env	n	Solver	SR (%)	ms / ep	ms / plan	ep $\times$	plan $\times$
Two-Room	50	GC-IDM	100.0 $\pm$ 0.0	260	86	1.0$\times$	1.0$\times$
		CEM	82.0 $\pm$ 2.0	10,486	10,317	40.3 $\times$	120 $\times$
		iCEM	88.0 $\pm$ 2.0	10,386	10,214	39.9 $\times$	119 $\times$
		MPPI	71.3 $\pm$ 6.4	2,994	2,823	11.5 $\times$	33 $\times$
		Gradient	34.0 $\pm$ 8.7	392	222	1.5 $\times$	2.6 $\times$
	200	GC-IDM	100.0 $\pm$ 0.0	276	398	1.0$\times$	1.0$\times$
		CEM	84.0 $\pm$ 2.8	10,502	41,318	38.1 $\times$	104 $\times$
		iCEM	87.5 $\pm$ 1.8	10,433	41,044	37.8 $\times$	103 $\times$
		MPPI	65.7 $\pm$ 2.0	3,022	11,400	10.9 $\times$	29 $\times$
		Gradient	30.3 $\pm$ 3.5	391	893	1.4 $\times$	2.2 $\times$
Push-T	50	GC-IDM	84.7 $\pm$ 5.0	305	86	1.0$\times$	1.0$\times$
		CEM	89.3 $\pm$ 6.4	10,558	10,340	34.6 $\times$	121 $\times$
		iCEM	84.7 $\pm$ 3.1	10,615	10,395	34.8 $\times$	121 $\times$
		MPPI	60.7 $\pm$ 4.2	3,162	2,940	10.4 $\times$	34 $\times$
		Gradient	0.7 $\pm$ 1.2	450	232	1.5 $\times$	2.7 $\times$
	200	GC-IDM	84.2 $\pm$ 2.8	322	397	1.0$\times$	1.0$\times$
		CEM	82.5 $\pm$ 1.3	10,778	42,232	33.5 $\times$	106 $\times$
		iCEM	80.0 $\pm$ 2.3	10,637	41,657	33.0 $\times$	105 $\times$
		MPPI	61.3 $\pm$ 2.3	3,176	11,809	9.9 $\times$	30 $\times$
		Gradient	2.5 $\pm$ 1.3	456	938	1.4 $\times$	2.4 $\times$
OGB-Cube	50	GC-IDM	99.3 $\pm$ 1.2	8,587	84	1.0$\times$	1.0$\times$
		CEM	73.3 $\pm$ 9.0	19,491	10,799	2.3 $\times$	129 $\times$
		iCEM	76.0 $\pm$ 7.2	19,880	11,047	2.3 $\times$	132 $\times$
		MPPI	49.3 $\pm$ 9.5	11,598	2,833	1.4 $\times$	34 $\times$
		Gradient	32.0 $\pm$ 3.5	8,360	229	1.0 $\times$	2.7 $\times$
	200	GC-IDM	98.7 $\pm$ 0.6	9,172	334	1.0$\times$	1.0$\times$
		CEM	67.0 $\pm$ 2.1	20,057	43,201	2.2 $\times$	130 $\times$
		iCEM	70.5 $\pm$ 4.0	20,616	44,652	2.2 $\times$	134 $\times$
		MPPI	49.3 $\pm$ 2.1	12,171	11,376	1.3 $\times$	34 $\times$
		Gradient	32.0 $\pm$ 4.8	8,948	654	1.0 $\times$	2.0 $\times$
Reacher	50	GC-IDM	100.0 $\pm$ 0.0	1,365	93	1.0$\times$	1.0$\times$
		CEM	68.0 $\pm$ 9.2	12,031	10,745	8.8 $\times$	116 $\times$
		iCEM	67.3 $\pm$ 11.4	11,693	10,482	8.6 $\times$	113 $\times$
		MPPI	42.7 $\pm$ 4.6	4,113	2,906	3.0 $\times$	31 $\times$
		Gradient	4.7 $\pm$ 1.2	1,481	239	1.1 $\times$	2.6 $\times$
	200	GC-IDM	99.7 $\pm$ 0.3	1,604	399	1.0$\times$	1.0$\times$
		CEM	70.3 $\pm$ 4.3	12,484	43,969	7.8 $\times$	110 $\times$
		iCEM	69.8 $\pm$ 0.3	11,871	41,790	7.4 $\times$	105 $\times$
		MPPI	45.7 $\pm$ 3.8	4,328	11,616	2.7 $\times$	29 $\times$
		Gradient	6.5 $\pm$ 2.6	1,685	937	1.1 $\times$	2.3 $\times$

Two-Room saturates at 100% by budget 25 and remains flat through 100. Reacher climbs to 99.5% by budget 50 and holds. OGBench-Cube peaks at 99.5% at budget 25 and degrades mildly to 96% at budget 100. Push-T is the notable case: it peaks at 93% at budget 25 and drops to 75% at budget 100, consistent with the h_{frac} clamping effect—at budgets exceeding $H_{\text{max}}=50$, the first steps receive a clamped horizon signal that was not seen during training.

E.8 Held-Out Validation

To rule out any form of trajectory-level memorization, we retrain GC-IDM on a 90%/10% episode-level split (10% of episodes are deterministically excluded at training time, seed 42) and evaluate both methods *exclusively on the held-out episodes*. Table K reports the results. Held-out success rates are statistically indistinguishable from in-distribution performance (Table 1) on every environment:

Table H: **GC-IDM architecture hyperparameter ablation.** Default configuration is hidden 512, 3 layers (boldface). Two-Room and Reacher are fully saturated across every setting. Push-T is the informative environment: hidden dimension is flat from 128 to 1024 (81–84%); depth matters more, with a monotone trend from 1 layer (70.5%) through 5 layers (87.5%). The default 3-layer configuration is within 2.5% of the best observed setting and was selected for compute efficiency.

Hidden dim	128	256	512 (default)	1024	
Two-Room	100.0	100.0	100.0	100.0	
Push-T	82.5	82.0	84.0	81.5	
OGB-Cube	98.5	99.0	99.0	99.0	
Reacher	99.5	99.5	99.5	99.5	
Layers	1	2	3 (default)	4	5
Two-Room	100.0	100.0	100.0	100.0	100.0
Push-T	70.5	80.5	85.0	86.5	87.5
OGB-Cube	97.5	99.5	99.0	96.5	100.0
Reacher	99.5	99.5	99.5	99.5	99.5

Table I: **Uniform noise schedule for GC-IDM.** Sampling σ per-batch from $[0, \sigma_{\max}]$ does not recover any of the performance lost to fixed- σ injection. Push-T sits in a narrow 78–80.5% band that is 4–7 points below the no-noise baseline (85.5%) and shows no preference between fixed and uniform schedules. The conclusion that GC-IDM does not need noise augmentation is robust to both schedule choice and environment.

$\sigma / \sigma_{\max}$	Fixed σ			Uniform $\sigma \sim [0, \sigma_{\max}]$		
	0.05	0.10	0.20	0.05	0.10	0.20
Two-Room	100.0	100.0	100.0	100.0	100.0	100.0
Push-T	80.0	80.5	79.5	80.5	80.5	78.0
OGB-Cube	99.0	98.5	98.5	99.0	98.5	98.5
Reacher	99.5	99.5	99.5	99.5	99.5	99.5

Two-Room remains at 100.0%, Push-T at 84.8% (vs. 84.2% in-distribution), OGBench-Cube at 98.5% (vs. 98.7%), and Reacher at 99.8% (vs. 99.7%). The negligible gap confirms that GC-IDM generalizes to unseen episodes and does not rely on memorizing specific trajectories from the training set. Speedup ratios on held-out episodes are also consistent with the main results, ranging from 2.2 $\times$ wall-clock on OGBench-Cube to 40 $\times$ on Two-Room.

Table K: **Held-out validation.** GC-IDM is retrained on 90% of episodes and evaluated exclusively on the 10% held-out partition; CEM is evaluated on the same held-out episodes. GC-IDM is reported as mean $\pm$ standard deviation across three training seeds; CEM is a single run per environment.

Env	Success (%)		ms / ep.		ms / plan		Speedup	
	Ours	CEM	Ours	CEM	Ours	CEM	wall	plan
Two-Room	100.0$\pm$0.0	86.0	279	11041	399	43476	40 $\times$	109 $\times$
Push-T	84.8$\pm$1.6	83.5	321	10574	397	41408	33 $\times$	104 $\times$
OGB-Cube	98.5$\pm$0.5	72.0	9248	20187	333	43548	2.2 $\times$	131 $\times$
Reacher	99.8$\pm$0.3	59.5	1542	13331	399	47564	8.6 $\times$	119 $\times$

Table J: **Success rate vs. evaluation budget.** Two-Room saturates at 100% by budget 25 and remains flat to budget 100. Push-T peaks at 93% at budget 25 and mildly degrades to 75–85% at larger budgets. Reacher climbs to 99.5% at budget 50 and holds through 100.

Budget (steps)	5	10	15	25	50	75	100
Two-Room	55.5	92.5	99.5	100.0	100.0	100.0	100.0
Push-T	11.5	28.0	59.0	93.0	84.5	77.5	75.0
OGB-Cube	43.0	49.0	68.0	99.5	99.0	97.0	96.0
Reacher	48.5	94.0	97.5	98.0	99.5	99.5	99.5

F More Implementation Details

IDM architecture. Both the pairwise and goal-conditioned IDMs use 3 hidden layers with dimension 512, LayerNorm, GELU activation, and 10% dropout. Weights are initialized with Kaiming normal; the output head uses a small initialization ($\sigma=0.01$) to start near zero.

Training hyperparameters. All IDMs are trained with AdamW ($\text{lr} = 10^{-3}$, weight decay = 10^{-4}), cosine annealing to $\text{lr}/100$, batch size 1024, for 50 epochs. Gradient clipping at norm 1.0 is applied. All other experiments sample evaluation episodes uniformly from the full dataset, matching the LeWM benchmark protocol.

Goal-conditioned IDM specifics. The maximum goal horizon $H_{\max}$ is set to 50 for all environments, matching the evaluation budget. During training, goal frames are sampled uniformly from $[1, H_{\max}]$ steps ahead within the same episode. The horizon embedding uses 64-dimensional sinusoidal encoding.

Embedding extraction. Frames are encoded in batches of 256 through the frozen LeWM encoder with ImageNet normalization. DataParallel is used when multiple GPUs are available. The HDF5 dataset is streamed to avoid loading all frames into memory.

Evaluation. We use the `stable_worldmodel` evaluation infrastructure with 200 parallel environments (LeWM’s hydra config uses 50; we use a larger sample for lower variance). The planning protocol matches LeWM’s `config/eval/{env}.yaml` exactly: start states are random and goal states are offset by 25 steps, and the agent has a 50-step budget. For the CEM baseline, we instantiate the solver via:

```
swm.solver.CEMSolver(model=cost_model, num_samples=300, var_scale=1.0,
                     n_steps=30, topk=30, device=device, seed=42)
```

with `swm.PlanConfig(horizon=5, receding_horizon=5, action_block=5)`. This matches the numerical defaults in `config/eval/solver/cem.yaml` and `config/eval/{env}.yaml` from `lucas-maes/le-wm`. Planning time is measured wall-clock including all encoding and inference.

Environment-specific configurations. We present the per-environment IDM training configuration in Table L.

Table L: IDM training configuration per environment.

	Two-Room	Push-T	Cube	Reacher
Embed dim (d)	192	192	192	192
Action dim	2	2	5	2
Frameskip	1	1	1	1
Hidden dim	512	512	512	512
n layers	3	3	3	3
Noise σ (GC)	0.0	0.0	0.0	0.0
$H_{\max}$	50	50	50	50
IDM params	$\sim 1.5\text{M}$	$\sim 1.5\text{M}$	$\sim 1.5\text{M}$	$\sim 1.5\text{M}$

G More Limitations and Future Works

In this paper, we focused mainly on a single world model backbone (LeWM) and *stable-worldmodel* benchmark suite. Three extensions remain open, with in-depth experiments showing the feasibility and strong performance of GC-IDM. In the future, we will expand our work toward: (i) Cross-world-model generalization: JEPA-style world models with different representation shapes, such as DINO-WM producing N -patch spatial features rather than a single CLS token, require a non-trivial architecture change to GC-IDM. (ii) Higher-dimensional action spaces: our four environments span 2–5 DoF, and benchmarks such as LIBERO with 7 DoF would test whether an MLP suffices at higher dimensionalities. (iii) Reactive-policy limits: GC-IDM does not reason about multi-step consequences, so for tasks with severe irreversibility a hybrid scheme using GC-IDM as a fast proposal with CEM as an occasional corrector would be a natural extension.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state that GC-IDM matches or exceeds CEM in 7/8 environment–protocol cells at 100–130× lower per-call planning cost, which is directly supported by Table 1.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: We discuss the limitation and future work in the Conclusion and Appendix.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [\[N/A\]](#)

Justification: We do not have theoretical contributions.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [\[Yes\]](#)

Justification: Appendix F reports all hyperparameters (optimizer, learning rate, batch size, epochs, architecture), per-environment configurations (Table L), evaluation protocol details, and the exact CEM solver instantiation. The pretrained LeWM checkpoints and datasets are publicly available from [Maes et al. \[2026\]](#).

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[No\]](#)

Justification: Code will be released upon acceptance. The paper uses publicly available LeWM checkpoints and datasets from [Maes et al. \[2026\]](#), and all implementation details needed for reproduction are provided in Appendix F.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Section 5.1 and Section 5.2 describe the experimental setup and compute accounting. Appendix F provides full details: AdamW optimizer with $\text{lr} = 10^{-3}$, batch size 1024, 50 epochs, cosine annealing, gradient clipping, the held-out split procedure, and per-environment configurations (Table L).

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: Table 1 reports mean $\pm$ standard deviation across three training seeds (42, 123, 456) for both GC-IDM and CEM. The caption explicitly states that error bars are seed standard deviations. Ablation studies use a single seed (42) and state this. The solver-family comparison (Table G) also reports mean $\pm$ std across three seeds.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 5.2 specifies GPU types (NVIDIA L4 for Two-Room, Push-T, Reacher; NVIDIA A100 for OGBench-Cube), GC-IDM training time (~20 minutes per environment on a single GPU), and per-episode wall-clock timing for both methods.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: This work studies planning efficiency in simulated benchmark environments. It does not involve human subjects, sensitive data, or dual-use concerns.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This is foundational research on planning efficiency in simulated control environments (navigation, manipulation, reaching). There is no direct path to negative societal applications beyond the general dual-use considerations that apply to all robotics research.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released asset is a small (1.5M-parameter) MLP for simulated control tasks. It poses no risk for misuse.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: LeWM [Maes et al., 2026] benchmark is cited throughout.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The GC-IDM architecture, training procedure, and hyperparameters are fully documented in Section 4.2 and Appendix F. Code will be released upon acceptance with reproduction scripts.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This work does not involve crowdsourcing or human subjects.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This work does not involve human subjects.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not a component of the core method. The method is a small MLP trained by supervised regression on frozen world-model embeddings.